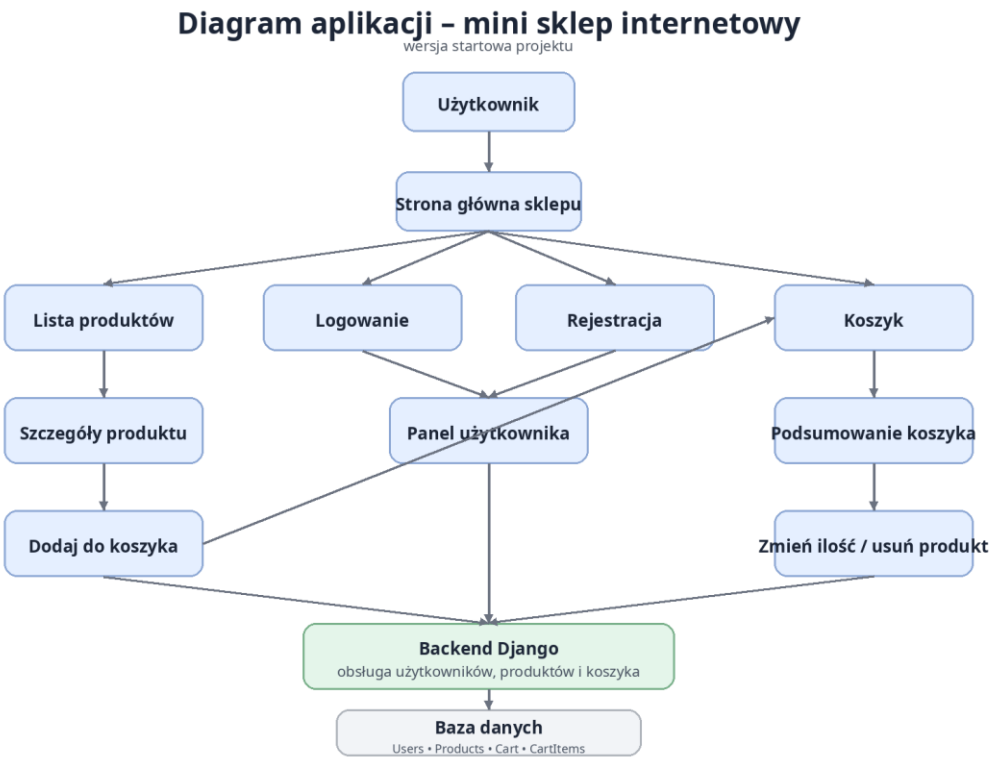
Mini sklep internetowy

Dokument koncepcyjny projektu i diagram startowy

Typ projektu	Aplikacja webowa w Django
Wersja dokumentu	Wersja startowa / MVP
Zakres	Produkty, konto użytkownika, koszyk
Cel	Stworzenie prostego i czytelnego sklepu internetowego do demonstracji pełnego przepływu użytkownika



Rysunek 1. Ogólny diagram działania aplikacji.

1. Opis projektu

Projekt zakłada stworzenie mini sklepu internetowego w Django. Użytkownik będzie mógł przeglądać produkty, zakładać konto, logować się, dodawać produkty do koszyka oraz zarządzać zawartością koszyka. Aplikacja ma być prosta, czytelna i możliwa do zrealizowania w warunkach projektu studenckiego.

2. Problem i cel biznesowy

Celem projektu jest pokazanie pełnego przepływu działania sklepu internetowego w małej skali. Aplikacja ma umożliwić wygodne przeglądanie oferty i zebranie produktów w koszyku po zalogowaniu. Projekt stanowi dobrą bazę do dalszego rozwoju o składanie zamówień, panel administratora i płatności.

3. Zakres funkcjonalny MVP

Moduł	Zakres	Priorytet
Konto użytkownika	Rejestracja, logowanie, wylogowanie, podstawowy panel użytkownika.	Niski
Produkty	Lista produktów, szczegóły produktu, prezentacja ceny, opisu i zdjęcia.	Wysoki
Koszyk	Dodawanie produktu do koszyka, zmiana ilości, usuwanie produktu, podsumowanie.	Wysoki
Warstwa wizualna	Przejrzysty układ strony głównej, listy produktów i formularzy.	Niski
Panel administracyjny	Dodawanie i edycja produktów z poziomu Django admin.	Średni

4. Główne widoki aplikacji

- Strona główna sklepu z wejściem do oferty, logowania i koszyka.
- Lista produktów z możliwością przejścia do szczegółów konkretnego produktu.
- Widok szczegółów produktu z przyciskiem dodania do koszyka.
- Logowanie i rejestracja użytkownika.
- Koszyk z możliwością zmiany ilości lub usuwania pozycji.
- Panel użytkownika z podstawowymi informacjami o koncie.

5. Założenia techniczne

- Backend: Python i framework Django.
- Frontend: HTML, CSS oraz szablony Django.
- Baza danych: SQLite na etapie początkowym.

- Zarządzanie produktami: panel Django admin.
- Autoryzacja: wbudowany system użytkowników Django.

6. Proponowane modele danych

Model	Najważniejsze pola	Relacje / rola
User	username, email, password	wbudowany model użytkownika
Product	name, description, price, image,	reprezentuje produkt w sklepie
Cart	user, created_at	jeden koszyk przypisany do użytkownika
CartItem	cart, product, quantity	pozycja w koszyku łącząca produkt z koszykiem

7. Harmonogram prac

Tydzień	Zakres prac	Efekt
1	Ustalenie pomysłu, diagram, repozytorium, struktura projektu.	Gotowy plan startowy
2	Modele danych, panel admin, dodanie produktów.	Działająca baza i produkty
3	Rejestracja, logowanie, panel użytkownika.	Obsługa kont
4	Dodawanie do koszyka, zmiana ilości, usuwanie pozycji.	Koszyk działa
5	Poprawa wyglądu HTML i CSS, testy przepływu użytkownika.	Czytelny interfejs
6	Dokumentacja, poprawki, przygotowanie prezentacji projektu.	Wersja do oddania

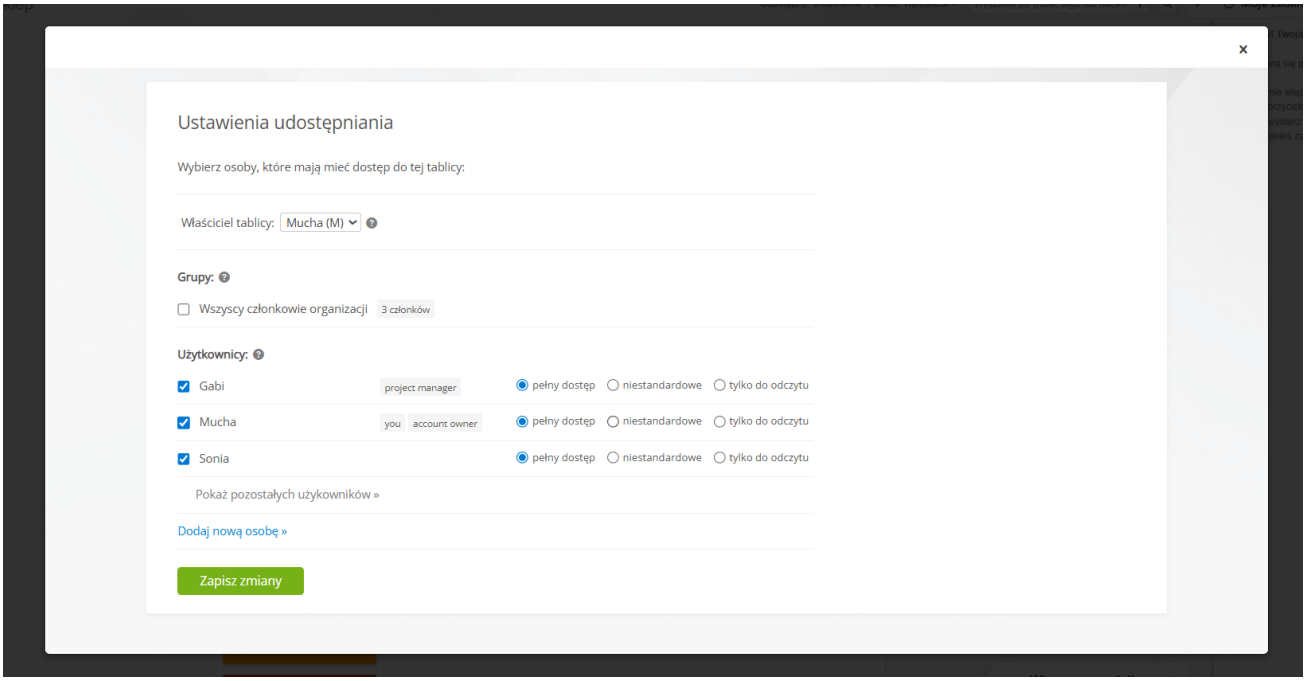
8. Ryzyka i ograniczenia

- Zbyt duży zakres projektu może utrudnić skończenie wszystkich funkcji w terminie.
- Na etapie MVP warto zrezygnować z płatności online i pełnej obsługi zamówień.
- Najważniejsze jest doprowadzenie do końca przepływu: użytkownik → produkt → koszyk.

9. Kierunek dalszego rozwoju

- Składanie zamówień i historia zamówień.
- Kategorie produktów i wyszukiwarka.
- Obsługa stanów magazynowych.
- Lepszy panel użytkownika i panel administratora.
- Integracja z płatnościami oraz wysyłką.

10. Kanban



Zespół wykorzystuje metodę Kanban do organizacji pracy nad projektem. Tablica składa się z następujących etapów:

• Do zrobienia – zadania zaplanowane do wykonania.
• W realizacji – zadania nad którymi aktualnie pracuje zespół.
• Testowane – zadania ukończone programistycznie i sprawdzane pod kątem poprawności działania.
• Gotowe – zadania zakończone i zaakceptowane przez zespół.

Każde zadanie przechodzi przez kolejne etapy aż do ukończenia.

11. WIP

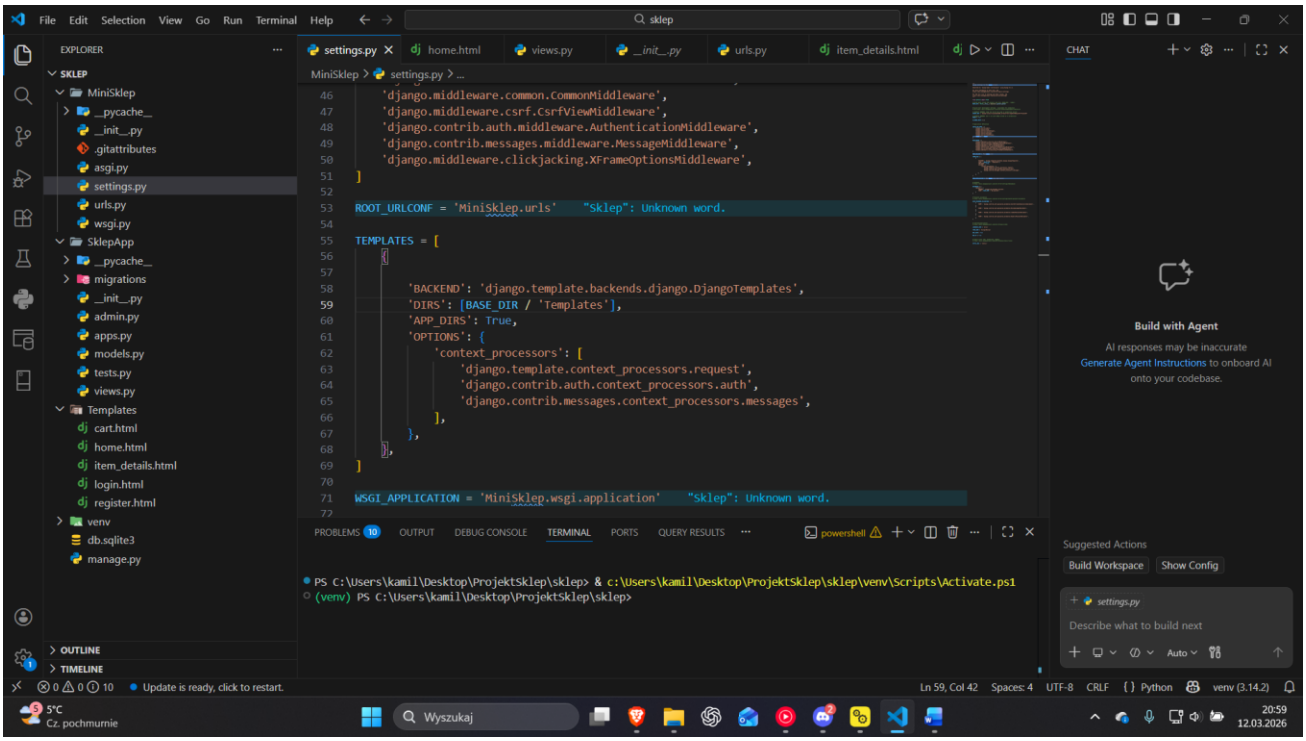
Aby ograniczyć pracę nad zbyt wieloma zadaniami jednocześnie, ustalono następujące limity:

- W realizacji – maksymalnie 3 zadania
- Testowane – maksymalnie 2 zadania

12. Team roles

Projekt realizowany jest przez trzyosobowy zespół.

(Kamil) - Backend developer odpowiada za logikę aplikacji i modele danych.
(Gabriela) - Frontend developer odpowiada za szablony HTML i stylowanie CSS.
(Sonia) - Trzecia osoba zajmuje się integracją funkcji (spójność backendu z frontendem) oraz testowaniem działania aplikacji.


settings.py
home.html
views.py
__init__.py
urls.py
item_details.html
MiniSklep > settings.py > ...
'django.middleware.common.CommonMiddleware',
'django.middleware.csrf.CsrfViewMiddleware',
'django.contrib.auth.middleware.AuthenticationMiddleware',
'django.contrib.messages.middleware.MessageMiddleware',
'django.middleware.clickjacking.XFrameOptionsMiddleware',
]
ROOT_URLCONF = 'MiniSklep.urls' "Sklep": Unknown word.
TEMPLATES = [
{
'BACKEND': 'django.template.backends.django.DjangoTemplates',
'DIRS': [BASE_DIR / 'Templates'],
'APP_DIRS': True,
'OPTIONS': {
'context_processors': [
'django.template.context_processors.request',
'django.contrib.auth.context_processors.auth',
'django.contrib.messages.context_processors.messages',
],
},
},
]
WSGI_APPLICATION = 'MiniSklep.wsgi.application' "Sklep": Unknown word.
PS C:\Users\kamil\Desktop\ProjektSklep\sklep> & c:\Users\kamil\Desktop\ProjektSklep\sklep\venv\Scripts\Activate.ps1
(venv) PS C:\Users\kamil\Desktop\ProjektSklep\sklep>
Build with Agent
AI responses may be inaccurate
Generate Agent Instructions to onboard AI onto your codebase.
Suggested Actions
Build Workspace
Show Config
Describe what to build next